

Diffusion-TS 完整原理笔记

一、前向过程：完全沿用经典 DDPM 的设计

从真实时间序列 x_0 开始，经过 T 步逐步添加噪声，使数据逐渐“脏化”，最终在第 T 步完全变成标准高斯噪声 x_T 。

加噪规则

- 每一步的加噪量由 β_t 控制（ β_t 是 0 到 1 之间的数值，例如 $\beta_t=0.01$ ，代表这一步添加 1% 的噪声）。
- 具体公式： $q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$
 - 公式翻译：第 t 步的带噪数据 x_t ，是从一个正态分布里抽样得到的——这个分布的中心（均值）是上一步数据 x_{t-1} 乘以 $\sqrt{1-\beta_t}$ （保证原始数据的特征不被完全覆盖），“分散程度”（方差）是 β_t （控制噪声大小）。

二、反向过程（Reverse Process）

模型学习从带噪序列 x_t 反向去噪声，逐步还原出与真实序列相似的 x_0 。针对时间序列的“可解释性”和“生成质量”，对反向过程的参数化方式和网络架构做了两处关键修改，未完全沿用经典 DDPM 的反向传播。

1. 编码器

将当前噪声序列 x_t 输入编码器，编码器采用 Transformer 的自注意力机制，捕捉不同时间步的关联，最终输出一个全局特征向量。

（1）第一步：纯噪声 x_t 进行位置编码

假设我们有 5 天的家电能耗纯噪声 $z=(0.72, 1.95, 0.38, 2.12, 0.55)$ （ $T=5$ 个时间步， $d=1$ 个特征），具体操作如下：

1. 生成位置编码：用正弦 / 余弦函数生成和噪声序列同维度的“时间标签”，简化后为[0.01, 0.02, 0.03, 0.04, 0.05]；
2. 加位置编码：将噪声和位置编码逐元素相加，得到 $a=[0.73, 1.97, 0.41, 2.16, 0.60]$ (5×1 的向量)。

(2) 第二步：通过线性层生成 Q、K、V 矩阵

1. 线性层的本质：“加权求和 + 偏移”，计算逻辑简化为公式：输出 = 输入 $\times$ W + b
 - 输入：带位置编码的噪声序列z（比如 5×1 的向量）；
 - W：线性层的“权重矩阵”（可训练参数，维度由“输入特征数”和“输出特征数”决定）；
 - b：线性层的“偏置向量”（可训练参数，用于微调输出，避免仅依赖加权）；
 - 输出：就是 Q、K、V 矩阵中的一个（比如 Q 矩阵）。
2. 关键特点：Q、K、V 各有一套独立的W和b（即 W_q 、 b_q 生成 Q， W_k 、 b_k 生成 K， W_v 、 b_v 生成 V），彼此不共享参数——这样能让 Q 专注于“提问题”，K 专注于“贴标签”，V 专注于“存信息”。

具体计算例子

步骤 1：确定线性层参数（训练后得到的权重和偏置）

假设经过训练，生成 Q 矩阵的线性层参数为：

- 权重矩阵 W_Q (1×2 ，因为输入特征数 = 1，输出特征数 = 2)： $\begin{bmatrix} 0.5 & 0.3 \end{bmatrix}$
- 偏置向量 b_Q (1×2)： $\begin{bmatrix} 0.05 & 0.1 \end{bmatrix}$

（注：这些参数不是随便设的，是模型通过训练数据学到的——比如模型发现“噪声数值 $\times 0.5 + 0.05$ ”能更好地提炼“查询信息”）

步骤 2：对每个时间步的噪声做线性变换

线性层会对a的每个时间步单独计算，得到 Q 矩阵的每一行：

- 第 1 天 $a[0]=0.73$ ：Q行 1 = $0.73 \times W_Q + b_Q = 0.73 \times \begin{bmatrix} 0.5 & 0.3 \end{bmatrix} + \begin{bmatrix} 0.05 & 0.1 \end{bmatrix} = \begin{bmatrix} 0.365 + 0.05 & 0.219 + 0.1 \end{bmatrix} = \begin{bmatrix} 0.415 & 0.319 \end{bmatrix}$
-

第 2 天 $a[1]=1.97$ ：Q行 2 = $1.97 \times \begin{bmatrix} 0.5, 0.3 \end{bmatrix} + \begin{bmatrix} 0.05, 0.1 \end{bmatrix} = \begin{bmatrix} 0.985 + 0.05, 0.591 + 0.1 \end{bmatrix} = \begin{bmatrix} 1.035, 0.691 \end{bmatrix}$

步骤 3：得到最终 Q 矩阵

将 5 个时间步的结果按行拼接，得到 5×2 的 Q 矩阵：

$$\begin{bmatrix} 0.415 & 0.319 \\ 1.035 & 0.691 \\ 0.223 & 0.255 \\ 1.13 & 0.748 \\ 0.35 & 0.28 \end{bmatrix}$$

（每行对应：第 1 天的查询、第 2 天的查询、第 3 天的查询、第 4 天的查询、第 5 天的查询）

同理生成 K 矩阵和 V 矩阵

最终得到 Q、K、V 矩阵，每个矩阵都是 5×2 ，具体数值如下表：

时间步	Query(Q)	Key(K)	Value(V)
1 (0.72)	[0.1, 0.2]	[0.08, 0.18]	[0.05, 0.15]
2 (1.95)	[0.3, 0.4]	[0.28, 0.38]	[0.25, 0.35]
3 (0.38)	[0.06, 0.12]	[0.05, 0.1]	[0.03, 0.12]
4 (2.12)	[0.4, 0.5]	[0.38, 0.48]	[0.35, 0.45]
5 (0.55)	[0.15, 0.25]	[0.12, 0.22]	[0.10, 0.20]

(3) 第三步：计算注意力权重 —— 判断谁和我相关

(1) 计算 “相似度得分” (Score)

用每个时间步的 Q 向量和所有时间步的 K 向量做 “点积”（对应元素相乘再相加），得到相似度得分。点积越大，说明 Q 和 K 越相似，两个时间步的关联越强。

例如计算 “第 1 天的 Q” 和所有时间步的 K 的得分：

- 第 1 天 Q × 第 1 天 K = $0.1 \times 0.08 + 0.2 \times 0.18 = 0.008 + 0.036 = 0.044$ ；
- 第 1 天 Q × 第 2 天 K = $0.1 \times 0.28 + 0.2 \times 0.38 = 0.028 + 0.076 = 0.104$ ；
- 第 1 天 Q × 第 3 天 K = $0.1 \times 0.06 + 0.2 \times 0.12 = 0.006 + 0.024 = 0.030$ ；

- 第 1 天 $Q \times$ 第 4 天 $K = 0.1 \times 0.38 + 0.2 \times 0.48 = 0.038 + 0.096 = 0.134$;
- 第 1 天 $Q \times$ 第 5 天 $K = 0.1 \times 0.12 + 0.2 \times 0.22 = 0.012 + 0.044 = 0.056$;

把所有得分整理成 “得分矩阵” (5×5 , 行对应 Q 的时间步, 列对应 K 的时间步) :

```
\begin{bmatrix}0.044 & 0.104 & 0.030 & 0.134 & 0.056 \\ \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots \end{bmatrix}
```

(2) 除以 “缩放因子” —— 避免得分过大

点积计算容易导致得分数值过大 (尤其是特征维度高时), 会影响后续 Softmax 函数的效果 (数值太大, Softmax 后会出现 “一个权重接近 1, 其他接近 0” 的极端情况)。所以需要除以 “ V 特征维度的平方根” (比如特征维度是 2, $\sqrt{2} \approx 1.414$), 让得分范围更合理。

例如第 1 天的得分缩放后:

```
0.044/1.414 \approx 0.031, 0.104/1.414 \approx 0.074, 0.030/1.414 \approx 0.021, 0.134/1.414 \approx 0.095, 0.056/1.414 \approx 0.040.
```

(3) Softmax 归一化 —— 把得分变成 “权重”

对缩放后的得分做 Softmax 处理, 让每个时间步的 “所有得分之和等于 1”, 这样得分就变成了 “注意力权重”, 权重越大, 说明该时间步的信息越重要, 需要重点关注。

例如第 1 天的权重计算 (Softmax 后): 权重 1 ≈ 0.18 , 权重 2 ≈ 0.21 , 权重 3 ≈ 0.17 , 权重 4 ≈ 0.24 , 权重 5 ≈ 0.20 (总和为 1)。这意味着: 第 1 天的噪声在融合信息时, 会重点关注第 4 天 (权重 0.24) 和第 2 天 (权重 0.21) 的信息, 因为它们的关联更强。

(4) 融合信息 —— 按权重取 “有用内容”

得到注意力权重后, 自注意力层会用 “权重” 去加权求和所有时间步的 Value (值) —— 权重高的 Value 贡献更多, 权重低的贡献更少, 最终得到每个时间步的 “注意力输出”。

实例 (第 1 天的输出计算) :

```
第 1 天的输出 = 权重 1  $\times$  第 1 天  $V$  + 权重 2  $\times$  第 2 天  $V$  + 权重 3  $\times$  第 3 天  $V$  + 权重 4  $\times$  第 4 天  $V$  + 权重 5  $\times$  第 5 天  $V$ 
```

```
= 0.18 \times [0.05, 0.15] + 0.21 \times [0.25, 0.35] + 0.17 \times [0.03, 0.12] + 0.24 \times [0.35, 0.45] + 0.20 \times [0.10, 0.20]
```

```
= \begin{bmatrix}0.009 + 0.0525 + 0.0051 + 0.084 + 0.02, 0.027 + 0.0735 + 0.0204 + 0.108 + 0.04\end{bmatrix}
```

$$= \begin{bmatrix} 0.1706, 0.2689 \end{bmatrix}$$

这个输出向量就融合了“所有时间步的信息”，尤其是第 4 天和第 2 天的 Value 贡献更多——相当于第 1 天的噪声“参考了全班同学的意见”，重点听了关联最强的同学的想法，得到了更全面的信息。

编码器最终输出

所有时间步都完成上述计算后，自注意力层会输出一个“ 5×2 ”的注意力输出矩阵，本质是融合了全局关联的中间特征。作用是给后续解码器的分解模块提供更优质的特征输入。

2. 解码器

解码器块的内部结构

Transformer 块 → 前馈网络（FNN）块 → 可解释层（趋势 / 季节层）

解码器的 Transformer 块

解码器的每个 Transformer 块都包含全注意力和交叉注意力，用于融合编码器信息。

第一个解码器块的输入：三大核心组成

- 组件 1：编码器输出的“全局关联信息”（Encoder Output）
 - 这是第一个解码器块最关键的“外部全局信息源”，也是所有解码器块共有的输入。
- 组件 2：预处理后的“noisy 时间序列特征”（ x_t 的特征映射）
 - 这是第一个解码器块的“初始局部特征源”，替代了“前一个解码器块的输出”（因为第一个块没有前序块）。
 - 通过 1×1 卷积（文档图 2 中的“Conv 1×1 ”）将 x_t 的原始特征维度（ $\tau \times d$ ）调整为和解码器内部特征维度一致（比如从 $d=7$ 调整为 256），方便后续注意力计算。
 - 位置编码：给每个时间步添加“位置信息”（文档附录 E 提到的“transformer sinusoidal positional embedding”）——因为时间序列的“顺序”至关重要（比如早 8 点和晚 8 点的用电负荷含义不同），位置编码能让模型识别时间先后关系。
- 组件 3：扩散步 t 的“嵌入信息”（Diffusion Step Embedding）
 - 这是扩散模型特有的输入，用于适配“不同步 t 的噪声程度差异”，文档附录 E 专门提到了这一设计。

- 将扩散步 t （比如 $t=500$ 中的某一步）通过“正弦位置编码 + 线性变换”转换为特征向量（形状： $1 \times \text{内部特征维度}$ ），再通过“自适应层归一化”（Adaptive Layer Normalization）注入到第一个解码器块的特征中。

输入的融合过程：第一个解码器块如何“整合三大组件”

1. 步骤 1：融合“预处理特征”和“扩散步嵌入”

- 通过自适应层归一化 ($a_t \cdot \text{LayerNorm}(w) + b_t$)，将扩散步 t 的嵌入信息注入到“预处理后的 x_t 特征”中，得到“带扩散步信息的局部特征 W ”。
- 目的：让局部特征适配当前去噪阶段的需求。

2. 步骤 2：全注意力层处理 W （捕捉局部依赖）

- 全注意力层计算 W 内部所有时间步的关联（如早 8 点和晚 7 点的高峰依赖），输出“融合局部依赖的特征 W_{self} ”（文档 3.2 节的 $w_{\text{tr}}^{\text{it}}$ 就是从这里衍生的趋势专用特征）。
- 目的：挖掘时间序列的局部时序规律。

3. 步骤 3：交叉注意力层融合“ W_{self} ”和“编码器全局信息”

- 交叉注意力层以 W_{self} 为 Q （查询），以编码器输出的全局信息为 K （键）和 V （值），计算“局部特征对全局信息的注意力权重”，再加权求和得到“融合全局信息的特征 W_{cross} ”。
- 目的：让局部特征符合全局趋势，避免偏离。

4. 步骤 4：残差连接 + 归一化

- 将交叉注意力层的输出 W_{cross} 与步骤 1 的 W 相加（残差连接，保证训练稳定），再经过层归一化，得到第一个解码器块的最终输出——这个输出会直接传给下一个解码器块的 FNN 层，或作为第一个块的可解释层（趋势 / 季节层）输入。

通过“全注意力 + 交叉注意力”捕捉了局部依赖和全局关联 → 传入 FNN 块。

步骤 1：前馈网络（FFN）细化特征

注意力层输出的 5×2 矩阵会先输入前馈网络——通过两层线性变换 + ReLU 激活函数，进一步提炼每个时间步的特征（比如把“低数值噪声关联”转化为更明确的“趋势特征信号”）。

例子：注意力层输出的某行 $[0.1706, 0.2689]$ ，经过前馈网络后可能变成 $[0.19, 0.29]$ ，特征更聚焦于趋势 / 季节规律。

步骤 2：解码器分解模块生成三组件

细化后的特征会输入解码器的趋势合成层和傅里叶合成层，同时分别生成 “趋势 ($V_{tr,t}$)、季节 ($S_{i,t}$)、最后一个解码器会输出误差 (R) ”。

1. 趋势 ($V_{tr,t}$)：

- 通过多项式回归生成 5×1 的平滑趋势向量（如[1.1, 1.3, 1.2, 1.5, 1.4]）；
- 第 1 个解码器块 ($i=1$)：捕捉 “短期局部趋势”；
- 第 2 个解码器块 ($i=2$)：捕捉 “长期全局趋势”；
- 求和得到最终趋势 $V_{tr,t}$ ；
- 混淆点：这里解码器块不是并行的，所有解码器块的 “局部趋势” 会在 “所有块都处理完后求和”，得到最终的全局趋势 $V_{tr,t}$ ；
- 补充：第一个解码器的 w_{tr} 是该块 FNN 的输出，通过 “ $C \cdot \text{Linear}(w_{tr}) + \text{块均值}$ ” 处理后，所有块结果累加得到趋势；第二个解码器的 w_{tr} 依赖上一个解码器块的输出（含上一层 FNN 的结果）。

2. 季节 ($S_{i,t}$)：

- 通过傅里叶变换生成 5×1 的周期向量（如[0.03, 0.10, -0.02, 0.09, -0.01]）；
- 第一个解码器块：会同时输出局部趋势、局部季节性组件。

3. 第二个解码器输入：3 个组件的融合

- 第一个块的加工特征：第一个块输出的 “通用特征”（已包含短期趋势 / 季节的关联信息）；
- 编码器的全局信息：始终传递的全局规律（如 “夏季整体用电高”），确保所有块不偏离全局；
- 扩散步 t 的嵌入信息：适配当前去噪阶段（如 t 较小时，优先细化细节）；
- 这三个组件融合后，作为第二个块的初始特征，进入其内部的 “Transformer→FNN→可解释层” 流程，提取该块的 “局部趋势（季节尺度）” 和 “局部季节（多周期融合）”。

4. 误差 (R)：

- 由最后一层解码器输出 5×1 的随机小波动向量（如[-0.01, 0.01, 0.01, -0.01, 0.01]）。

步骤 3：三组件融合得到最终预测值 $\hat{x}_0$

将趋势、季节、误差三组件逐元素相加，得到 5×1 的最终预测序列 $\hat{x}_0$ （比如[1.12, 1.40, 1.19, 1.58, 1.40]）—— 这才是和真实时间序列 x_0 （如家电能耗真实数据）对比的 “预测值”。

三、反向传播计算损失

1. 步骤 1：计算傅里叶总损失 L_{θ}

损失函数包含“时间域损失”和“频率域损失”两部分，公式回顾： $L_{\theta} = w_t \cdot [\text{mathcal{L}}_{\text{time}}] + \lambda \cdot \|\text{FFT}(\hat{x}_0) - \text{FFT}(x_0)\|_2^2$

- 时间域损失：计算 $\hat{x}_0$ 与真实序列 x_0 的数值差异（如 $x_0=[1.2, 1.5, 1.3, 1.6, 1.4]$ ， $\hat{x}_0=[1.12, 1.40, 1.19, 1.58, 1.40]$ ，差异通过均方误差计算）；
- 频率域损失：对 x_0 和 $\hat{x}_0$ 做 FFT，计算频率分布差异（确保周期性一致，比如“每天周期”的振幅是否匹配）；
- w_t ：权重，让模型更关注噪声多的扩散步骤（如 $t=400$ 比 $t=10$ 的权重更大）。

2. 步骤 2：反向传播计算参数梯度

损失值会通过“反向传播算法”，从“最终预测值 $\hat{x}_0$ ”反向推导到“最开始的注意力层参数”，计算每个参数（如 W_Q, b_Q ）对损失的“贡献度”（即梯度）：

例子：如果损失过大是因为“注意力层对第 1 天和第 3 天的关联权重太小”，反向传播会计算出 W_Q 的梯度，指示“需要调整 W_Q 的某个元素，让 Q_1 和 K_3 的相似度得分更高”。

3. 步骤 3：更新所有可训练参数

根据计算出的梯度，用优化器（如 Adam）更新模型的所有参数，包括：

- 注意力层参数： $W_Q, b_Q, W_K, b_K, W_V, b_V$ （让下次注意力权重更准确捕捉关联）；
- 解码器参数：趋势 / 季节合成层的多项式系数、傅里叶变换权重（让三组件更贴近真实规律）；
- 前馈网络参数：线性层权重和偏置（让特征细化更有效）。

4. 步骤 4：迭代训练直到损失收敛

重复“前向计算（从噪声 x_t 到 $\hat{x}_0$ ）→ 计算损失→ 反向传播→ 更新参数”的流程，直到损失值稳定下降到较低水平（比如从初始的 0.5 降到 0.01）—— 此时模型学会了从噪声中准确恢复真实时间序列的规律。

四、训练结束后，进入采样阶段

反向过程核心公式

$$p(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$$

- 公式含义：这个正态分布是用来“描述去噪后 x_{t-1} 的概率分布”——比如“去噪后 x_{t-1} 最可能落在 μ_{θ} 附近，波动范围是 Σ_{θ} ”。

经典 DDPM 的理想后验分布（附录 B 公式 18）

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}(x_t, x_0), \tilde{\beta}_t)$$

- 理想后验均值 $\tilde{\mu}(x_t, x_0)$ ：若知道真实的 x_0 ，去噪后的 x_{t-1} 最可能围绕“均值 $\tilde{\mu}(x_t, x_0)$ ”波动，波动范围由方差 $\tilde{\beta}_t$ 决定；
- 物理意义：均值 $\tilde{\mu}(x_t, x_0)$ 平衡了两个信息源： x_0 （干净数据的真实结构）和 x_t （当前带噪数据的时序依赖）。

Diffusion-TS 的适配：用 $\hat{x}_0$ 替代 x_0

- 经典 DDPM 的理想后验均值依赖“已知真实 x_0 ”，但在训练 / 采样阶段，我们并不知道真实 x_0 （采样时甚至是从纯噪声开始，完全没有 x_0 ）；
- Diffusion-TS 的核心训练目标是“让模型直接预测 $\hat{x}_0$ ”（3.3 节：“train the neural network to predict an estimate $\hat{x}_0$ directly”），因此自然地用模型预测的 $\hat{x}_0$ 替代理想后验均值中的真实 x_0 ，得到公式中的 μ_{θ} （模型预测的后验均值）；
- 这一步是公式设计的“核心适配”——既保留了 DDPM 的数学严谨性，又契合 Diffusion-TS “直接学习干净数据结构”的训练目标。

公式中参数的作用（动态调整权重）

公式中的 α_t 和 β_t （由 α_t 衍生）不是随意设定的，它们的作用是动态调整 x_0 （干净结构）和 x_t （时序依赖）的权重，确保去噪后的 x_{t-1} 既符合真实数据结构，又保持时序连贯性。用“扩散步 t 的不同阶段”举例：

- 当 t 很大（接近 T ，噪声多）： $\alpha_t \approx 0$
 - 此时 x_t 几乎是纯噪声，包含的“时序依赖”信息极少，需要更多依赖 x_0 的干净结构；

- 代入参数后， $\mu_{\theta} \approx \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \hat{x}_0$ ($\hat{x}_0$ 权重较大)， $\frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$ (x_t 权重较小)；
- 物理意义：t 大时， x_{t-1} 主要由 $\hat{x}_0$ 的结构决定（比如先还原股票的长期趋势）， x_t 的噪声信息仅起辅助作用。

2. 当 t 很小（接近 1，噪声少）： $\alpha_t \approx 1$

- 此时 x_t 接近干净数据，包含的“时序依赖”信息丰富，不需要过多依赖 $\hat{x}_0$ ；
- 代入参数后， $\mu_{\theta} \approx \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \hat{x}_0$ ($\hat{x}_0$ 权重较小)， $\frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$ (x_t 权重较大)；
- 物理意义：t 小时， x_{t-1} 主要由 x_t 的时序信息决定（比如微调股票的短期波动）， $\hat{x}_0$ 仅用于修正少量噪声。

随机项系数的意义

随机项的系数 $\sigma_{\theta}(x_t, t) = \sqrt{\tilde{\beta}_t}$ 也不是随意的——它正好是 DDPM 后验分布方差的平方根（附录 B 公式 18 的方差项为 $\tilde{\beta}_t$ ）。

这意味着：随机项的波动范围与“当前扩散步的噪声水平”匹配——t 大时方差大（随机波动强，允许更多结构探索），t 小时方差小（随机波动弱，仅微调细节），进一步保证生成的连贯性。

混淆点： $\hat{x}_0$ 在训练阶段和采样阶段的区别

阶段	$\hat{x}_0$ 的含义	作用
训练阶段	真实数据集的固定 x_0 （有标准答案）	计算傅里叶损失，调整模型参数
采样阶段	当前对应的潜在 x_0 （无标准答案）	代入 x_{t-1} 公式，生成下一步带噪数据；通过多步迭代间接贡献到最终 x_0

（注：文档部分内容可能由 AI 生成）